Certicom-131 rho step-rate parity on one CPU core

Study `research/certicom131_rho_parity_20261023/`, built on the merge commit `701931104`. Run window 2026-10-23, 7 interleaved rounds. This study follows the 2026-10-22 baseline (`research/certicom131_rho_ic_bench_20261022/`), which measured ECC2-131 at 8.5 M and ECCp-131 at 1.2 M steps/s against 13 M for the ECC2K-130 client. Every number is labelled **measured**, **counted** or **derived**; raw records are in `raw/`, every value in `results.json`.

Summary

- **Measured (one pinned core, same session, 59 isolation records, none contended).** The ECC2K-130 client runs at **10.3 M iterations/s** (two positions in the round, 11.1 and 10.2 M; their ratio 0.92 is the A/A drift and the noise floor of this host). On the exact ECCp-131 curve the new AVX-512 walk reaches **15.1 M steps/s** (1.46x the client, **11.8x** the 32-bit arm re-measured in the same session). On the exact ECC2-131 curve the bitsliced walk on the client's own field code reaches **11.5 M steps/s** (1.11x the client, within the A/A drift, so parity and not a demonstrated lead; 1.27x the old scalar carry-less-multiply arm).
- **Negative results kept.** A scalar 3x64-bit Montgomery arm for ECCp-131 reaches only 4.9 M steps/s (0.48x the client). My restructured scalar ECC2-131 arm with 3 carry-less products per multiply (`clmul3`) is **slower** than the old 9-product arm: 6.9 M against 9.0 M steps/s (0.77x). Interleaving independent inversion chains and advancing lanes in lock-step did not help either scalar arm.
- **Derived.** At these rates the expected rho work is about 6.2 thousand core-years for ECC2K-130, 90 thousand for ECC2-131 (was 115 thousand with the old arm) and 73 thousand for ECCp-131 (was 866 thousand), on this host class. These are projections from a step rate, not solves; the host differs from the 2026-10-22 study (2.8 GHz Xeon here, 2.1 GHz there), so absolute rates are not comparable across the two studies; only the in-session ratios are.
- **What parity means here.** Equal rho steps per second per CPU core with the ECC2K-130 client. It does not equalise the expected work: ECC2-131 and ECCp-131 still need 16.2x and 17.3x the steps (counted, baseline study). The GPU claims are untouched: no CUDA path exists for either new target.
- **Index calculus: no improvement this round.** The gap to the matched rho (3.4x to 75x on the scaled ladders) is unchanged; section "Index calculus" records which phases carry it. Parity of index calculus with rho is an open research problem, not reached here.
- **Classification (AGENTS.md section 3).** Rho step rate: engineering. No IC row, no leaderboard or `benchmarks/README.md` row (validation gate not met).

Method and instruments

New programs, all in this directory, all checked against the header arithmetic `gpu/certicom131/certicom131.cuh` through independent scalar conversions:

- `pv131.cpp`, ECCp-131: 8 walks per AVX-512 register, 5 limbs of 27 bits (`vpmuludq`, lazy carries, Montgomery R = 2^135), Montgomery's trick across 64 registers (one inversion chain per lane), per-lane table lookup by `vpermt2q`, class rule "even canonical y".
- `bs131.cpp`, ECC2-131: the curve constants are moved once into the normal-basis coordinates of the ECC2K-130 client (`generated/eccF131.h` change-of-basis tables); the walk then uses the client's bitsliced 512-lane multiplier, free squarings and Itoh-Tsujii inversion (`ecc2k130/include/fieldbs.h`), 16 slots per inversion, 16-way table select through minterm masks, class rule "first set coordinate of x".
- `walk131.cpp`: scalar arms (3x64 Montgomery; 3-product carry-less multiply) with `chains` interleaved inversion chains. Successor of the baseline study's `bench131.cpp`, which stays unchanged and is the "old" arm below.

Walk and unpriced work, for every arm: j chosen from x only, P becomes canon(P + T[j]) with the negation map, 16-entry table of random combinations of G and Q. The timed step includes table selection, denominators, the batch inverse, the affine addition and canonicalisation. It does not carry jump-scalar bookkeeping or a distinguished-point test or any storage; those are unpriced on every arm, the ECC2K-130 client's bench mode included (AGENTS.md section 5a), and reported as missing, not zero.

Gates (a nonzero exit on any failure; all passed in the isolated gated runs `raw/gate_*.stdout`): G1 field oracle (`pv131`: 8,192 products and 512 inversions; `bs131`: 10,240 products, 1,024 squares, 1,024 inversions; `walk131`: 20,000 products per arm), 0 mismatches; G2 step oracle, every lane after one iteration equals plus or minus the header's affine add (`pv131` 512 lanes, `bs131` 4,096 lanes, `walk131` 368 pairs per arm), 0 mismatches; G3 end points on the curve (the timing runs of both vector arms also check 32 sampled lanes after the run); G4 trajectories identical across slot counts (16, 32, 64 slots and 8, 16, 32 slots); zero zero-denominator events in the gated and timing runs of the scalar arms.

Protocol. All timed commands ran on reserved CPU 3 through `tools/isolated_bench.py` (59 records, 0 contended). Order per round: ECC2K-130 client, old prime arm, old binary arm, new scalar prime arm, new scalar binary arm, `pv131`, `bs131`, ECC2K-130 client again; seven rounds, so the first and last runs of each round give the A/A pair. `bs131` prints three samples per process (21 in all). One gated command (`gate_walk131`) was refused by the isolation gate because my own session was active during its settle window; it was re-run unchanged (recorded in `raw/exit_codes.txt`).

An incident, kept. The first attempt (`raw_attempt1_degenerate_start/`) reported a nonzero exceptional count for the bitsliced arm: one lane had been started exactly on a table point (my start generator returned the first point when two sampled base points had equal x, instead of doubling), so its denominator was zero every other step. It was a setup defect, not an arithmetic fault (lanes are independent and the speed was unaffected), but a run with exceptional events is not clean. The generators now use the proper add,

reject infinity and reject any start whose x equals a table point's; the frozen run was repeated into a fresh `raw/`, and the first attempt is retained and not used for any number below.

Host (recorded). Intel Xeon @ 2.80 GHz, 4 vCPU, KVM guest, AVX-512 F/DQ/BW/VL, BMI2, PCLMULQDQ, no VPCLMULQDQ, no IFMA, no PMU, no GPU; L2 1 MiB per core, L3 33 MiB; g++ 13.3 with `-O3 -march=native`.

Results

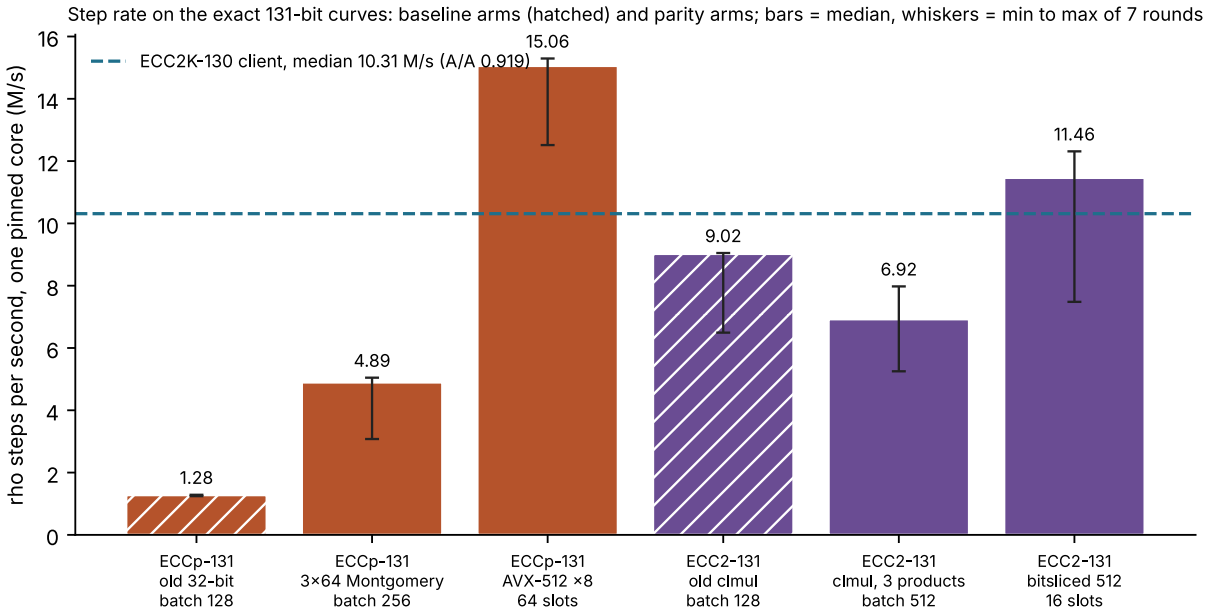


Figure 1: Measured rho steps per second on one pinned core, 7 rounds (`bs131`: 21 samples), bars are medians, whiskers the min to max. Hatched: the baseline arms re-measured in the same session. The dashed line is the ECC2K-130 client's median over both positions.

arm (one core)	median M steps/s	min to max	ratio to client	vs old arm
ECC2K-130 client, position 1	11.08	9.67 to 11.70	1.08	
ECC2K-130 client, position 8 (A/A)	10.19	9.34 to 12.06	0.99	
ECCp-131 old, 32-bit, batch 128	1.28	1.24 to 1.29	0.12	1.0
ECCp-131 3x64 Montgomery, batch 256	4.89	3.08 to 5.05	0.47	3.84
ECCp-131 AVX-512 x8, 64 slots	15.06	12.51 to 15.29	1.46	11.8
ECC2-131 old clmul, batch 128	9.02	6.49 to 9.05	0.87	1.0
ECC2-131 clmul3, batch 512	6.92	5.25 to 7.98	0.67	0.77
ECC2-131 bitsliced 512, 16 slots	11.46	7.48 to 12.31	1.11	1.27

Table 1: Measured. Ratio to client uses the median of both client positions, 10.31 M/s. The A/A ratio of the two client positions is 0.919. Differences below that drift (about 8%) are not read as differences; the 1.11 for ECC2-131 therefore means parity, the 1.46 for ECCp-131 is outside it. The ECC2-131 bitsliced spread (7.5 to 12.3) is wide: one of 21 samples falls below 9 M/s (7.48) and four below 10 M/s.

Phase profile of the bitsliced step (measured in the gated run, per 512-lane slot): multiply 6.9 us (13.5 ns per lane), table select 1.4 us, canonicalisation 0.25 us, whole inversion 55.9 us (8 multiplies). Five multiplies plus the select and one sixteenth of an inversion predict about 40 us, or 80 ns per lane-step; the measured median step is 87 ns.

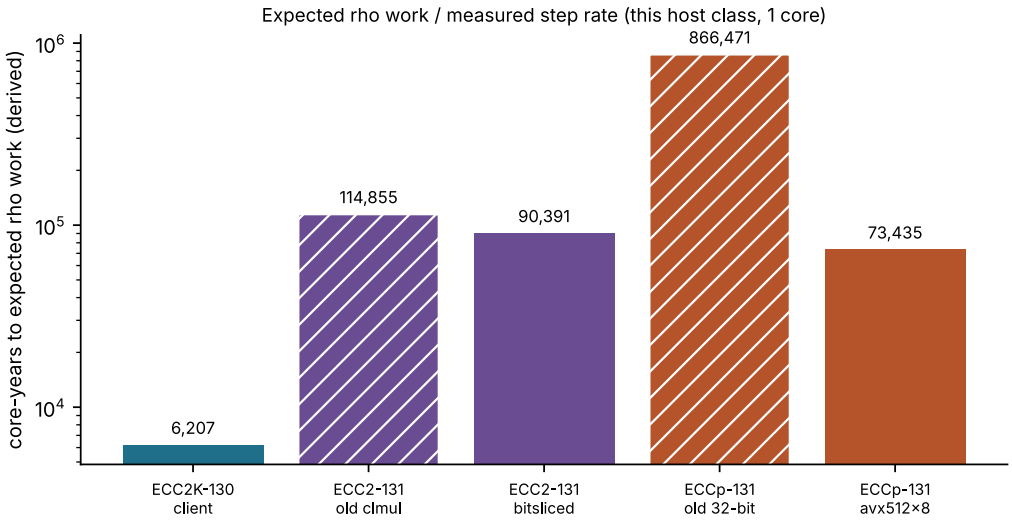


Figure 2: Derived: 2^(log2 expected steps) / measured median rate / 31,557,600 s, with the counted expected work of the baseline study (2^60.81, 2^64.83, 2^64.92). Hatched: the baseline arms, same-session rates.

Checks against the table in the request. With these rates the ECC2-131 to ECC2K-130 core-year ratio is 14.6x (steps 16.2x counted, rate 1.11x the client's) and the ECCp-131 ratio 11.8x (steps 17.3x counted, rate ratio 1.46). The request's "about 15x" for ECC2-131 now holds in time as well as in steps on this host class. The "about 21x" for ECCp-131 is not reproduced in time on CPU: with a vector step faster than the Koblitz client's iteration it is 11.8x here; on GPU it can only be measured with a CUDA port.

Index calculus

No index-calculus change was made. For the record, the phase split of the best variant per instance at the two largest ladder sizes of the baseline study (`ic boundary --seed 131`, group-addition equivalents, shares of the variant's own total): prime 2^21.1 and 2^23.9: factor base 37% and 44%, relations 62% and 56%; random binary 2^22 and 2^26: factor base 16% and 58%, relations 84% and 42%; Koblitz 2^27.8 and 2^39: factor base 75% and 71%, relations 25% and 29%. Linear algebra is below 1% everywhere. So at the measured sizes the gap to the matched rho (3.4x to 75x) is carried by building the factor base and finding relations, in comparable shares; the fitted total exponents (baseline study) stay above rho's. This is a diagnostic. Closing the gap needs a lower-cost relation or factor-base method on identical instances, verified and priced end to end; none was found or claimed here.

What this study does not show

- No GPU result. These are single-core CPU rates for walks without distinguished-point handling; a full solve needs DP storage, collision handling and verification on top.
- The ECC2K-130 client counts its own iteration function (Frobenius orbit weight); its rate and the new arms' rates are equal-work-per-step only in the sense of one affine addition per step, not in progress per step. The expected-work ratios are the counted ones.
- One host, one session (7 rounds). A/A drift 8%; `bs131` has a wide spread. Rates on other hosts were not measured; the 2026-10-22 study's host was slower.
- The fastest external implementation for these two curves was not run (external comparison: not run); the 2009 published figures are the only external reference.
- No `benchmarks/README.md` or leaderboard row: the validation gate (independent replay, validator report, matched rho with the 0 / 0.1 / 1 GAE band) is not met.

Next improvements to try

1. A CUDA port of the ECCp-131 batched walk on the same limb scheme, and of the ECC2-131 walk on the client's packed kernel, gated against these programs.
2. Add the distinguished-point test and jump-scalar bookkeeping to the timed step, so the unpriced work is priced on every arm.
3. Reduce the ECC2-131 bitsliced spread (check slot count, memory layout) and test an AVX-512 `vpermt2q` mux against the minterm mux.
4. For index calculus: a relation oracle or factor-base construction that lowers the 56% to 84% relation and 16% to 75% factor-base shares on identical instances, with end-to-end verification.

Files

`walk131.cpp`, `pv131.cpp`, `bs131.cpp`, `pf64.h`, `run_parity.sh` (frozen command list), `analyze_parity.py`, `results.json`, `fig1_step_rates.svg/png`, `fig2_core_years.svg/png`, `raw/` (59 isolation records, gated and timing outputs, `exit_codes.txt`), `raw_attempt1_degenerate_start/` (first attempt, retained).